

Lab Report No 9
Stepper Motor Control
Microprocessor and Interfacing techniques



Submitted By:

Huzaifa Shahbaz

21-CE-009

Submitted to:

Engr. Bushra Fiaz

Dated:

Week 10

Department of Computer Engineering,

HITEC University, Taxila

Lab Task No 1:

Solution:

Brief description (3-5 lines)
Use the eight slide switches to set the speed of rotation of the stepper motor shaft from 0 to 31.875 RPM in steps of 0.125 RPM. Write a program in which BTND will switch the motor operation from FULL step to HALF step when depressed. BTNR will switch the rotational direction of the rotor from CW to CCW when pressed. The speed of rotation will be displayed on the four-digit seven-segment display digits with 25% duty cycle persistence and one ms update rate.
The code
<pre>#pragma config FSRSEL = PRIORITY_7 // Shadow Register Set Priority Select (SRS Priority 7) #pragma config PMDL1WAY = ON // Peripheral Module Disable Configuration (Allow only one reconfiguration) #pragma config IOL1WAY = ON // Peripheral Pin Select Configuration (Allow only one reconfiguration) // DEVCFG2uvf #pragma config FPLLIDIV = DIV_2 // PLL Input Divider (2x Divider) #pragma config FPLLMUL = MUL_20 // PLL Multiplier (20x Multiplier) #pragma config FPLLODIV = DIV_1 // System PLL Output Clock Divider (PLL Divide by 1) // DEVCFG1 #pragma config FNOSC = PRIPLL // Oscillator Selection Bits (Fast RC Osc w/Div-by-N (FRCDIV)) #pragma config FSOSCEN = OFF // Secondary Oscillator Enable (Enabled) #pragma config IESO = ON // Internal/External Switch Over (Enabled) #pragma config POSCMOD = XT // Primary Oscillator Configuration (Primary osc disabled) #pragma config OSCIOFNC = OFF // CLKO Output Signal Active on the OSCO Pin (Disabled) #pragma config FPBDIV = DIV_8 // Peripheral Clock Divisor (Pb_Clk is Sys_Clk/8) #pragma config FCKSM = CSDCMD // Clock Switching and Monitor Selection (Clock Switch Disable, FSCM Disabled) #pragma config WDTPS = PS1048576 // Watchdog Timer Postscaler (1:1048576)</pre>

```

#pragma config WINDIS = OFF           // Watchdog Timer Window Enable
(Watchdog Timer is in Non-Window Mode)
#pragma config FWDTEN = OFF           // Watchdog Timer Enable (WDT
Disabled (SWDTEN Bit Controls))
#pragma config FWDTWINSZ = WINSZ_25   // Watchdog Timer Window
Size (Window Size is 25%)
// DEVCFG0
#pragma config DEBUG = OFF             // Background Debugger Enable
(Debugger is Disabled)
#pragma config JTAGEN = ON             // JTAG Enable (JTAG Port Enabled)
#pragma config ICESEL = ICS_PGx1      // ICE/ICD Comm Channel Select
(Communicate on PGEC1/PGED1)
#pragma config PWP = OFF               // Program Flash Write Protect (Disable)
#pragma config BWP = OFF               // Boot Flash Write Protect bit (Protection
Disabled)
#pragma config CP = OFF                // Code Protect (Protection Disabled)

#include <sys/attribs.h>
#include <xc.h>

#define A1 LATBbits.LATB3             //LAT, TRIS and PORT registers can be
defined by a keyword for convenience
#define A2 LATEbits.LATE8
#define B1 LATEbits.LATE9
#define B2 LATBbits.LATB5
#define MODE LATFbits.LATF1

void delay_ms(int);                   //Blocking Delay Function for x millisecond,
using Timer 2
void revo_half_step(int);              //This Function rotates the stepper motor for x
revolution in half step mode
void revo_full_step(int);              //This Function rotates the stepper motor for x
revolution in full step mode

int main(void)
{
    TRISFbits.TRISF1 = 0;              //set RF1 (MODE) to be an output
    TRISBbits.TRISB3 = 0;              //set RB3 (AIN1) to be an output

```

```

    ANSELBbits.ANSB3 = 0;           //disable analog functionality for RB3
(AIN1)
    TRISEbits.TRISE8 = 0;           //set RE8 (AIN2) to be an output
    TRISEbits.TRISE9 = 0;           //set RE9 (BIN1) to be an output
    TRISBbits.TRISB5 = 0;           //set RB5 (BIN2) to be an output
    ANSELBbits.ANSB5 = 0;           //disable analog functionality for RB5
(BIN2)

    MODE=0;

    revo_half_step(1);               //the function's argument is "number of
    revolutions to be performed"
    while(1);
}

void delay_ms(int ms)               //Blocking Delay Function for x millisecond,
using Timer 2
{
    T2CONbits.TON = 0;               //turn OFF Timer 2
    T2CONbits.TCKPS = 7;             //pre-scale = 1:256 (T2CLKIN or freq_clk =
39062.5 Hz)
    PR2 = 39*ms;                     //T2 period ~ 20mS. As Time = (1/fclk_clk)*PR
    TMR2 = 0;                         //clear Timer 2 counter *
    T2CONbits.TON = 1;               //turn OFF Timer 2
    while(IFS0bits.T2IF==0);
    IFS0bits.T2IF = 0;
}

void revo_full_step(int x)          //This Function rotates the stepper motor for x
revolution
{
    int cnt=0;
    int rev=0;
    while(rev<=x-1)                  //This while loop is to perform x revolutions
    {
        cnt=0;
        while(cnt<=199)              //This while loop is to perform one revolution
        {
            A1=1;
            B1=0;

```

```

    A2=0;
    B2=0;
    delay_ms(2);
    A1=0;
    B1=1;
    A2=0;
    B2=0;
    delay_ms(2);
    A1=0;
    B1=0;
    A2=1;
    B2=0;
    delay_ms(2);
    A1=0;
    B1=0;
    A2=0;
    B2=1;
    delay_ms(2);
    cnt++;
    }
    rev++;
}

}

void revo_half_step(int x)           //This Function rotates the stepper motor for x
revolution
{
    int cnt=0;
    int rev=0;
    while(rev<=x-1)                  //This while loop is to perform x revolutions
    {
        cnt=0;
        while(cnt<=199)              //This while loop is to perform one revolution
        {
            A1=1;
            B1=1;
            A2=0;
            B2=0;
            delay_ms(2);

```

```
A1=0;
B1=1;
A2=0;
B2=0;
delay_ms(2);
A1=0;
B1=1;
A2=1;
B2=0;
delay_ms(2);
A1=0;
B1=0;
A2=1;
B2=0;
delay_ms(2);
A1=0;
B1=0;
A2=1;
B2=1;
delay_ms(2);
A1=0;
B1=0;
A2=0;
B2=1;
delay_ms(2);
A1=1;
B1=0;
A2=0;
B2=1;
delay_ms(2);
A1=1;
B1=0;
A2=0;
B2=0;
delay_ms(2);
cnt++;
}
rev++;
}
```

}

The results (Screenshot)

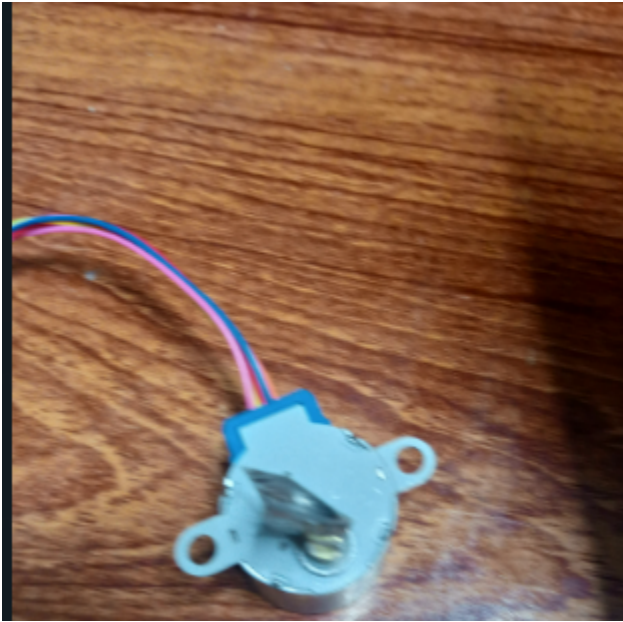


FIGURE 9.1: STEPPER MOTOR

Conclusion:

In conclusion, stepper motor control is a precise and efficient method for regulating motion in various applications. Its ability to move in discrete steps makes it suitable for tasks requiring accuracy, while the ease of control through electronic pulses enhances its versatility. Overall, stepper motors offer a reliable solution for applications demanding controlled and synchronized movement.